

PROG3/B: Einführung in die Programmierung mit C++

Prof. Dr.-Ing. Enrico Schröder

01.07.2024

Table of contents

Vorwort	4
Aufbau	4
Inhalte	4
Verwendung	4
Copyright	5
1 Einführung	6
1.1 Von C zu C++: Hello World	7
1.2 Kompilieren von C++-Code	7
2 Von C zu C++	9
2.1 Strukturen	9
2.2 Ein- und Ausgabe	11
2.3 Referenzen	11
2.4 Namespaces	13
2.5 Der Datentyp <code>string</code>	14
2.5.1 <code>stringstream</code>	14
3 Klassen	16
3.1 Deklaration vs. Definition	16
3.2 Sichtbarkeit und Zugriff	17
3.3 Objekte	19
3.4 Konstruktoren	19
3.4.1 Der <code>this</code> -Zeiger	20
3.4.2 Verwendung von Konstruktoren	21
3.4.3 Überladung	21
3.4.4 Die Initialisierungsliste	22
3.4.5 Der default-Konstruktor	22
3.4.6 Der Kopierkonstruktor	23
3.5 Destruktoren	24
3.6 <code>const</code> -Methoden	25
3.7 Operatorüberladung	25
4 Klassenhierarchien und Vererbung	27
4.1 Eigenschaften von Klassen	27
4.2 UML-Klassendiagramme	28

4.3	Vererbung	28
4.3.1	Auswirkungen von Vererbung	31
4.3.2	Sichtbarkeiten bei Vererbung	31
4.3.3	Virtuelle Methoden und Polymorphie	32
4.3.4	Typecasting	34
4.3.5	Abstrakte Klassen	36
4.4	Modultechnik	36
5	Die Standardbibliothek	38
5.1	Container	38
5.1.1	<code>std::vector</code>	39
5.1.2	<code>std::list</code>	40
5.1.3	Iteratoren	41
5.1.4	<code>std::map</code>	41
5.2	Algorithmen	42
5.2.1	<code>std::sort</code>	43
5.2.2	<code>std::find</code>	44
6	Dynamischer Speicher	45
6.1	Statischer vs. dynamischer Speicher	45
6.2	Speicher allozieren und freigeben	46
6.3	Smart Pointer	46
6.3.1	Ownership bzw. <i>wer besitzt das Objekt?</i>	47
6.3.2	<code>std::shared_ptr</code>	48
6.3.3	<code>std::weak_ptr</code>	49
6.3.4	<code>std::make_unique</code> und <code>std::make_shared</code>	51
6.3.5	<code>std::dynamic_pointer_cast</code> und <code>std::static_pointer_cast</code>	51
7	Templates	52
7.1	Was sind Templates?	52
7.2	Template-Klassen	53
7.3	Template-Funktionen	54
8	Verschiedenes	55
8.1	Exceptions	55
8.1.1	Exceptions in C++	55
	References	57

Vorwort

Dieses Skript ist Teil der Veranstaltung **Programmieren 3/Teil B** an der Fakultät EFI der Technischen Hochschule Nürnberg (OHM).

Aufbau

Es werden Grundlagen der Programmiersprache C++ vermittelt. Da es sich um eine Veranstaltung mit relativ geringem Umfang handelt, ist der Anspruch des Skripts *nicht*, C++ in seiner vollen Tiefe zu behandeln, sondern stattdessen die wichtigsten Konzepte und Techniken zu vermitteln, die für die praktische Anwendung notwendig sind. Für einen tieferen Einblick in die Sprache wird auf weiterführende Literatur verwiesen, wie z.B. (Mahr 2023).

Es wird davon ausgegangen, dass die Studierenden bereits Erfahrungen sowohl der Sprache C als auch mit Objektorientierung (z.B. Java) haben.

Inhalte

Der Fokus des Skripts sind die grundlegenden Konzepte der Programmiersprache C++:

- Unterschiede zu C
- Klassen
- Klassenhierarchien
- Algorithmen und Container
- Templates
- Exceptions
- Smart Pointer

Verwendung

Dieses Skript ist als HTML-Seite sowie PDF verfügbar. Es wird dazu geraten, zuerst die HTML-Seite zu lesen, da diese mit einem Index sowie Suchfunktion versehen ist. Sämtliche Anwendungsbeispiele sind lauffähiger C++ Code. Die einfachste Möglichkeit, diesen auszuführen,

ist die Verwendung eines Online-Compilers, wie z.B. [OnlineGDB](#). Sie sind explizit dazu eingeladen, die Codebeispiele selbst auszuführen und mit ihnen zu experimentieren. Um das noch weiter zu erleichtern, finden Sie in der HTML-Version dieses Skripts neben den Codebeispielen jeweils einen “Kopieren”-Button.

Copyright

Prof. Dr.-Ing. Enrico Schröder, Professor für maschinelles Lernen, Fakultät EFI, Technische Hochschule Nürnberg (OHM)

Verwendung zu Studienzwecken erlaubt. Vervielfältigung oder Weitergabe ausgeschlossen.

1 Einführung

C++ ist eine direkte Erweiterung der Programmiersprache C. Wie C auch, ist C++ eine **statisch typisierte** Sprache, die **kompiliert** wird. Das bedeutet, dass der Code von einem **Compiler** in Maschinencode übersetzt wird, bevor er ausgeführt werden kann und dass sämtliche Variablen bzw. Ausdrücke einen Datentypen haben, der während des Kompilierens feststehen und überprüfbar sein muss.

C++ ist eine **objektorientierte** Programmiersprache, die Klassen und Objekte verwendet, um die Modellierung von komplexen Problemen zu erleichtern.

Klassen und Objekte sind die zentralen Konzepte der objektorientierten Programmierung. Klassen repräsentieren häufig reale Objekte oder Akteure und definieren, welche Eigenschaften und Funktionen diese haben. Objekte sind konkrete Instanziierungen von Klassen und können auf die Eigenschaften und Funktionen der Klasse zugreifen.

Zum Beispiel könnte eine Klasse **Fahrrad** definiert werden, die die Eigenschaften **Farbe**, **Marke** und **Modell** sowie die Funktionen **fahren()**, **bremsen()** und **reparieren()** hat. Ein Objekt dieser Klasse wäre dann ein konkretes Fahrrad.

Eine **Klasse** definiert und kapselt Eigenschaften und Funktionen. Ein **Objekt** ist eine Instanz einer Klasse.

In diesem Zusammenhang sind außerdem die Konzepte **Abstraktion**, **Kapselung**, **Polymorphie** und **Vererbung** zu nennen (diese sind allgemeingültig und bei jeder objektorientierten Sprache anzutreffen, nicht bloß bei C++):

- Abstraktion: Eine Klasse im Code *abstrahiert* ein reales Objekt oder einen realen Akteur, indem es Schnittstellen bereitstellt, die definieren, wie bestimmte Funktionalitäten ausgeführt werden bzw. wie die Klasse mit anderen Klassen kommuniziert.
- Kapselung: Zusammengehörige Daten und Methoden werden in Klassen gebündelt und Zugriff auf sie nur über festgelegte Schnittstellen erlaubt.
- Polymorphie: Unterschiedliche Klassen können dieselbe Schnittstelle implementieren und auf unterschiedliche Weise auf diese Schnittstelle reagieren.
- Vererbung: Klassen können von einander erben und dadurch erweitert bzw. spezialisiert werden.

1.1 Von C zu C++: Hello World

Unten ist ein einfaches Programm in C dargestellt, das den Text “Hello World” auf der Konsole ausgibt.

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello World\n");
5     return 0;
6 }
```

Das gleiche Programm in C++ sieht so aus:

```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Hello World" << std::endl;
5     return 0;
6 }
```

Sie sehen, dass die beiden Programme sehr ähnlich sind. Ein paar Unterschiede sind jedoch zu beachten:

- Für die Eingabe- bzw. Ausgabefunktionen wird die Header-Datei `<iostream>` anstelle von `<stdio.h>` verwendet.
- Die Funktion `printf()` wird durch `std::cout` ersetzt.
- Der Operator `<<` wird verwendet, um Text auf der Konsole auszugeben.
- `std::endl` wird anstelle von `"\n"` für eine neue Zeile verwendet.

Wir werden uns mit all dem später noch genauer beschäftigen, aber das sollte Ihnen einen ersten Eindruck davon geben, wie sich C und C++ unterscheiden.

C++ ist eine Erweiterung von C. Gültiger C-Code ist auch gültiger C++-Code (mit wenigen Ausnahmen).

1.2 Kompilieren von C++-Code

Um C++-Code zu kompilieren, benötigen Sie einen C++-Compiler. In C haben Sie sicherlich `gcc` aus der *GNU Compiler Collection* verwendet, um Ihren Code zu kompilieren. In C++

können Sie `g++` verwenden ¹. Der Aufruf erfolgt genauso wie bei `gcc` im Terminal, nur dass Sie `g++` anstelle von `gcc` verwenden.

```
> g++ -Wall -o hello hello.cpp
```

Der obige Befehl kompiliert den Code in der Datei `hello.cpp` und erstellt eine ausführbare Datei namens `hello`. `-Wall` ist ein Flag, das den Compiler anweist, alle Warnungen auszugeben. Sie sollten Compiler-Warnungen immer ernst nehmen, da sie häufig auf potenzielle Fehler hinweisen.

Nutzen Sie beim compilieren **immer** das Flag `-Wall` und beheben Sie alle auftretenden Warnungen! Dies spart Ihnen viel Zeit und Ärger bei der Fehlersuche.

¹Die Standardbibliothek von C++ ist sehr umfangreich und enthält eine Vielzahl nützlicher Funktionen. Wir werden uns in Kapitel 5 näher mit ihr beschäftigen.

2 Von C zu C++

Als erstes werden wir uns einige Unterschiede zwischen C und C++ ansehen und dabei ein paar grundlegende Konzepte von C++ kennenlernen.

2.1 Strukturen

Auch in C gibt es schon die Möglichkeit, eigene Datentypen zu definieren und mittels `struct` Daten zu kapseln.

Schauen wir uns ein Beispiel an:

```
1 // C Code
2 typedef struct {
3     int x;
4     int y;
5 } Point;
6
7 void printPoint(Point p) {
8     printf("(%d, %d)\n", p.x, p.y);
9 }
10
11 int main() {
12     Point p;
13     p.x = 1;
14     p.y = 2;
15
16     printPoint(p);
17
18     return 0;
19 }
```

In C++ können wir das gleiche Konzept verwenden, Strukturen sind allerdings deutlich mächtiger.

Die Struktur Point könnte in C++ so aussehen:

```

1 struct Point {
2     int x;
3     int y;
4 };

```

Zusätzlich zu den Daten können wir in C++ auch Methoden in Strukturen gekapselt werden. In diesem Beispiel wird die Methode `printPoint()` zur Struktur selbst hinzugefügt:

```

1 struct Point {
2     int x;
3     int y;
4
5     void print() {
6         std::cout << "(" << x << ", " << y << ")" << std::endl;
7     }
8 };

```

Damit haben wir die inhaltlich zur Struktur passende `print`-Funktion direkt in der Struktur definiert und damit das reale Objekt *Punkt* noch besser *gekapstelt*. Innerhalb dieser Funktion können wir direkt auf die Daten der Struktur zugreifen.

Die deklaration der Struktur ist in C++ fast identisch zu C:

```

1 int main() {
2     Point p;
3
4     p.x = 1;
5     p.y = 2;
6
7     p.print();
8
9     return 0;
10 }

```

Zuerst wird ein Objekt `p` als Instanz der Struktur `Point` erstellt. Anschließend werden die Werte `x` und `y` des Punktes gesetzt.

Zum Aufrufen von Funktionen innerhalb einer Struktur wird der Punkt-Operator `.` verwendet. Mit `p.print()` wird die Methode `print` des Punktes `p` aufgerufen.

Funktionen innerhalb einer Struktur bzw. Klasse werden als **Methoden** bezeichnet.

2.2 Ein- und Ausgabe

In C wurde die Ausgabe mittels `printf()` realisiert. In C++ gibt es `std::cout`, welches in der Headerdatei `iostream` definiert ist.

`std::cout` ist keine Funktion, sondern ein s.g. Stream-Objekt. Aus diesem Grund verwendet man auch nicht etwa `std::cout("text")`, sondern den Stream-Operator `<<`. Ein Stream ist eine Sequenz von Bytes, die von einer Quelle zu einem Ziel fließen. Es gibt in C++ verschiedene Arten von Streams, die für unterschiedliche Zwecke verwendet werden können (z.B. Dateien, Konsolen, Netzwerkverbindungen, etc.). `cout` ist der Standardausgabestream, der auf die Konsole schreibt.

Der Operator `<<` wird verwendet, um Daten in den Stream zu schreiben. In welche Richtung die Daten fließen, lässt sich anhand des Operators intuitiv erkennen. Im Falle von

```
1 std::cout << "text" << std::endl;
```

fließen die Daten von rechts nach links, also zuerst vom String "text" in den Stream `std::cout`. Anschließend wird ein Zeilenumbruch (`std::endl`) in den Stream geschrieben.

Für die Eingabe gibt es den Stream `std::cin`, der ebenfalls in der Headerdatei `iostream` definiert ist. Der Operator `>>` wird verwendet, um Daten aus dem Stream zu lesen. Im Falle von

```
1 int x;  
2 std::cin >> x;
```

fließen die Daten von `std::cin` in die Variable `x`.

2.3 Referenzen

In C gibt es Zeiger, die verwendet werden, um auf Speicheradressen zu verweisen. Ein Anwendungsfall dafür ist *call-by-reference*, wobei eine Funktion eine Referenz auf eine Variable erhält und somit den Wert der Variablen ändern kann.

Schauen wir uns ein Beispiel an. Die Funktion `swap()` tauscht die Werte zweier Punkte aus:

```

1 // C Code
2 void swap(Point *a, Point *b) {
3     Point tmp = *a;
4     *a = *b;
5     *b = tmp;
6 }
7
8 int main() {
9     Point p1 = {1, 2};
10    Point p2 = {3, 4};
11
12    swap(&p1, &p2);
13
14    return 0;
15 }

```

Ein Problem bei der Verwendung von Zeigern ist, dass sie leicht zu Fehlern führen können. Zum Beispiel kann ein Zeiger auf eine ungültige Speicheradresse zeigen, was meist zum Absturz des Programms führt und unbedingt vermieden werden muss!

In C++ gibt es daher s.g. **Referenzen**, die eine sicherere Alternative zu Zeigern darstellen. Eine Referenz verhält sich wie ein Zeiger, kann aber nicht auf NULL zeigen und muss bei der Initialisierung auf ein konkretes Objekt bzw. eine Variable verweisen.

Eine Referenz wird mit dem &-Operator definiert, also z.B. `Point &a` anstatt `Point *a`.

Die Funktion `swap()` könnte in C++ wie folgt aussehen:

```

1 void swap(Point &a, Point &b) {
2     Point tmp = a;
3     a = b;
4     b = tmp;
5 }
6
7 int main() {
8     Point p1 = {1, 2};
9     Point p2 = {3, 4};
10
11    swap(p1, p2); // Kein &-Operator notwendig!
12
13    return 0;
14 }

```

Beachten Sie, dass beim Aufruf einer Funktion mit Referenzen kein `&`-Operator verwendet wird. Der Compiler weiß, dass es sich um eine Referenz handelt und übergibt automatisch die Adresse des Objekts.

Zwar gibt es in C++ auch Zeiger, aber in den meisten Fällen sind Referenzen die bessere Wahl.

Nutzen Sie für *call-by-reference* in C++ Referenzen statt Zeigern.

2.4 Namespaces

In C++ gibt es s.g. **Namespaces**, die verwendet werden, um den Gültigkeitsbereich von Funktionen, Klassen und Variablen zu definieren. Ein Namespace ist ein Bereich, in dem Namen eindeutig sein müssen. Das bedeutet, dass zwei Funktionen mit dem gleichen Namen in unterschiedlichen Namespaces existieren können, ohne dass es zu Konflikten kommt.

Ein Namespace wird mit dem `namespace`-Schlüsselwort definiert. Der Name des Namespaces wird in geschweiften Klammern angegeben. Innerhalb des Namespaces können Funktionen, Klassen und Variablen definiert werden.

In den obigen Beispielen haben Sie die Funktion `std::cout` gesehen. `std` ist solch ein Namespace, der Funktionen und Klassen der C++-Standardbibliothek ¹ beinhaltet.

Sie können auch eigene Namespaces definieren. Zum Beispiel könnte ein Namespace `geometry` definiert werden, der u.a. unsere Struktur `Point` enthält:

```
1 namespace geometry {  
2     struct Point {  
3         ...  
4     };  
5 }
```

In diesem Falle müssten wir auf die Struktur `Point` mit `geometry::Point` zugreifen.

Um den Code übersichtlicher zu gestalten, können Sie auch den `using namespace`-Befehl verwenden. Zum Beispiel sieht man häufig `using namespace std;` am Anfang einer cpp-Datei. Damit kann man dann wieder direkt (also ohne `std::...`) auf die Funktionen und Klassen des `std`-Namespaces zugreifen. Nutzen Sie dies jedoch nur in cpp-Dateien und nicht in Headerdateien, um Namenskonflikte beim inkludieren zu vermeiden!

¹Die Standardbibliothek von C++ ist sehr umfangreich und enthält eine Vielzahl nützlicher Funktionen. Wir werden uns in Kapitel 5 näher mit ihr beschäftigen.

2.5 Der Datentyp string

In C gibt es keinen Datentyp für Zeichenketten. Zeichenketten werden als Arrays von Zeichen dargestellt, wobei das letzte Zeichen per Konvention ein Nullzeichen ist und das Ende der Zeichenkette markiert.

In C++ gibt es den Datentyp `std::string`, der in der Headerdatei `string` definiert ist. Ein `std::string` ist ein Objekt, das eine Zeichenkette repräsentiert. Mittels des `=`-Operators können Zeichenketten einem String zugewiesen werden.

```
1  #include <iostream>
2  #include <string>
3
4  int main() {
5      std::string name = "Alice";
6
7      std::cout << "Hello, " << name << "!" << std::endl;
8
9      return 0;
10 }
```

Nutzen Sie in C++ den Datentyp `std::string` für Zeichenketten.

Die Klasse `std::string` bietet eine Vielzahl von nützlichen Methoden, z.B. um Zeichenketten zu manipulieren oder innerhalb des Strings zu suchen. Schauen Sie sich die [Referenz](#) an, um mehr über die Methoden zu erfahren.

2.5.1 stringstream

Um verschiedene Datentypen in einen String zu konvertieren, können Sie `std::stringstream` verwenden, welcher im Header `sstream` definiert ist. Ein `std::stringstream` ist ein Stream-Objekt, das Daten in einen String schreibt (sehen Sie die Parallele zu `std::cout`?). Sie können verschiedene Datentypen in den Stream schreiben und anschließend den gesamten Inhalt des Streams als String auslesen.

```
1  #include <iostream>
2  #include <string>
3  #include <sstream>
4
5  int main() {
6      std::stringstream ss;
7  }
```

```
8     int x = 42;
9     double y = 3.14;
10
11     ss << "x = " << x << ", y = " << y;
12
13     std::string result = ss.str();
14
15     std::cout << result << std::endl;
16
17     return 0;
18 }
```

3 Klassen

Wie in Kapitel 1 erläutert, sind Klassen ein zentrales Konzept der objektorientierten Programmierung.

Eine Klasse modelliert häufig einen realen Gegenstand oder eine abstrakte Entität bzw. einen abstrakten Akteur. In ihr werden die Eigenschaften und das Verhalten der Entität beschrieben und gebündelt.

In C++ werden Klassen mit dem Schlüsselwort `class` deklariert:

```
1 class Vector2D {  
2     // Attribute  
3     int x;  
4     int y;  
5  
6     // Methoden  
7     void print();  
8     void add(const Vector2D& otherVector);  
9     ...  
10 };
```

Innerhalb der Klasse definieren wir die Attribute, also alle Eigenschaften bzw. Variablen, die ein Objekt dieser Klasse haben kann. Außerdem deklarieren bzw. definieren wir die Methoden (also Funktionen), die das Verhalten der Klasse beschreiben.

Attribute und Methoden, die innerhalb einer Klasse definiert sind, werden auch als *Member* bezeichnet.

Die Methoden einer Klasse können auf die Attribute der Klasse zugreifen und diese verändern sowie auf andere Methoden der Klasse zugreifen.

3.1 Deklaration vs. Definition

An dieser Stelle ein kleiner Exkurs: In der Programmierung wird zwischen der *Deklaration* und der *Definition* von Funktionen unterschieden. Die Deklaration einer Funktion gibt an,

wie die Funktion aufgerufen werden kann, also wie viele und welche Parameter sie erwartet und welchen Rückgabewert sie hat. Die Definition einer Funktion gibt an, was die Funktion genau macht, also den Funktionsrumpf (das, was innerhalb der geschweiften Klammern {...} der Funktion steht).

Versuchen Sie, eine Funktion aufzurufen, die nur deklariert, aber nicht definiert ist, kommt es zu einem **Linker-Fehler**: Der Linker kann die Funktion nicht finden und bricht den Kompilervorgang ab.

Im obigen Beispiel haben wir bisher nur die Methoden der Klasse *deklariert*. Für die Definition müssen wir noch den Funktionsrumpf schreiben (man sagt auch *implementieren*). In **cpp** gibt es verschiedene Möglichkeiten, wo wir die Methoden einer Klasse definieren können. Wir werden uns zunächst auf die Definition innerhalb der Klasse beschränken. Dazu schreiben wir den Funktionsrumpf direkt unter die Deklaration der Methode:

```
1 class Vector2D {
2     // Attribute
3     int x;
4     int y;
5
6     // Methoden
7     void print() {
8         std::cout << "(" << x << ", " << y << ")" << std::endl;
9     }
10
11     void add(const Vector2D& otherVector) {
12         x += otherVector.x;
13         y += otherVector.y;
14     }
15     ...
16 };
```

3.2 Sichtbarkeit und Zugriff

Bei der Deklaration der Attribute und Methoden müssen wir angeben, ob diese *öffentlich* oder *privat* sind. Standardmäßig sind alle Attribute und Methoden einer Klasse **private**, d.h. sie sind nur innerhalb der Klasse sichtbar und können nur von Methoden der Klasse selbst genutzt werden. Eine Klasse mit ausschließlich privaten Mitgliedern ist jedoch meist nicht sehr nützlich... Um auf Attribute und Methoden einer Klasse von außen zuzugreifen, müssen wir diese explizit als **public** (öffentlich) deklarieren. Dies tun wir mit den entsprechenden Schlüsselwörtern **public:** und **private:** vor den jeweiligen Mitgliedern:

```

1 class Vector2D {
2 private:
3     int x;
4     int y;
5
6 public:
7     void print() {...};
8     void add(const Vector2D& otherVector) {...};
9     ...
10 };

```

Im Sinne der Abstraktion und Datenkapselung ist es empfehlenswert, die Attribute einer Klasse als **private** zu deklarieren. Diese sind meist interne Details der Klasse, die von außen nicht direkt verändert werden sollten. Ein Zugriff auf die Attribute sollte über Methoden erfolgen, die den Zugriff kontrollieren und ggf. validieren, s.g. *Getter* und *Setter*.

Im Falle unserer Klasse `Vector2D` könnten wir z.B. Getter- und Setter-Methoden für die Attribute `x` und `y` hinzufügen:

```

1 class Vector2D {
2 private:
3     int x;
4     int y;
5
6 public:
7     int getX() const {
8         return x;
9     }
10    int getY() const {
11        return y;
12    }
13    void setX(int newX) {
14        x = newX;
15    }
16    void setY(int newY) {
17        y = newY;
18    }
19    ...
20 }

```

Das Schlüsselwort **const** nach den Methoden deklariert, dass die Methode keine Attribute der Klasse verändern darf (siehe auch Kapitel [3.6](#)).

Attribute sollten immer **private** deklariert werden! Methoden hingegen können **public** oder **private** sein, je nachdem, ob sie von außen aufgerufen werden sollen oder nicht.

Wie Sie vielleicht gemerkt haben, unterscheiden sich Strukturen und Klassen in C++ nur in einem Punkt: Alle Member einer Struktur sind standardmäßig **public**, während sie bei Klassen standardmäßig **private** sind.

3.3 Objekte

Eine Klasse ist ein Bauplan für Objekte. Ein Objekt ist eine konkrete Instanz einer Klasse. Ein Objekt hat also die Eigenschaften und das Verhalten, das in der Klasse definiert ist. In C++ erzeugen wir Objekte einer Klasse, indem wir eine Variable vom Typ der Klasse deklarieren:

```
1  int main() {  
2      Vector2D v1;  
3      v1.setX(3);  
4      v1.setY(4);  
5  
6      Vector2D v2;  
7      v2.setX(1);  
8      v2.setY(2);  
9  
10     v1.add(v2);  
11     v1.print();  
12     ...  
13 }
```

Mit dem `.-`Operator können wir auf die Attribute und Methoden eines Objekts zugreifen (soweit die Sichtbarkeit es zulässt). In obigem Beispiel nutzen wir die Setter, um beiden Objekten unterschiedliche Werte zu geben und addieren den Vektor `v2` auf `v1` und geben dann das Ergebnis mit `print()` aus.

3.4 Konstruktoren

In obigem Beispiel haben wir die Attribute der Klasse `Vector2D` nicht initialisiert. Zwar könnten wir direkt bei der Deklaration der Attribute die initialen Werte angeben, jedoch gibt es in C++ eine weitaus mächtigere Möglichkeit, das Objekt bei der Erzeugung zu initialisieren: den *Konstruktor*.

Ein Konstruktor ist eine spezielle Methode einer Klasse, die automatisch aufgerufen wird, wenn ein Objekt der Klasse erzeugt wird. Der Konstruktor hat den gleichen Namen wie die Klasse und hat keinen Rückgabewert.

```
1 class Vector2D {
2     ...
3 public:
4     // Konstruktor
5     Vector2D() {
6         x = 0;
7         y = 0;
8     }
9     ...
10 };
```

In obigem Beispiel haben wir einen s.g. *default-Konstruktor* definiert, der die Attribute `x` und `y` auf 0 setzt.

Ein Konstruktor kann auch Parameter haben, um das Objekt bei der Erzeugung zu initialisieren. Zum Beispiel macht es Sinn, zu erlauben, die Werte des Vektors gleich bei der Initialisierung des Objekts anzugeben:

```
1     ...
2     // Konstruktor mit Parametern
3     Vector2D(int x, int y) {
4         this->x = x;
5         this->y = y;
6     }
```

3.4.1 Der `this`-Zeiger

Der `this`-Zeiger ist ein spezieller Zeiger, der auf das aktuelle Objekt zeigt. Er wird implizit in jeder Methode einer Klasse definiert und kann genutzt werden, um auf die Attribute des Objekts zuzugreifen. Falls die Variablennamen in der Funktion eindeutig sind, kann der `this`-Zeiger auch weggelassen werden. Im obigen Fall jedoch haben die Parameter des Konstruktors die gleichen Namen wie die Attribute, daher müssen wir den `this`-Zeiger nutzen, um eindeutig zu spezifizieren, dass wir die Attribute der Klasse verändern wollen.

3.4.2 Verwendung von Konstruktoren

Bei der Deklaration eines Objekts können wir nun den passenden Konstruktor aufrufen, um das Objekt zu initialisieren:

```
1 int main() {  
2     Vector2D v1(3, 4);  
3     Vector2D v2(1, 2);  
4     ...  
5 }
```

3.4.3 Überladung

In C++ können mehrere Funktionen gleichen Namens existieren, solange sie sich in der Anzahl oder den Typen der Parameter unterscheiden. Dies nennt man *Überladung*. Überladung ist nicht nur für Konstruktoren möglich, sondern für alle Funktionen.

Häufig wird es aber bei Konstruktoren genutzt, um unterschiedliche Initialisierungen zu ermöglichen. So könnten wir z.B. einerseits den default-Konstruktor und andererseits einen Konstruktor mit Parametern anbieten:

```
1 class Vector2D {  
2     ...  
3 public:  
4     // default-Konstruktor  
5     Vector2D() {  
6         x = 0;  
7         y = 0;  
8     }  
9  
10    // Konstruktor mit Parametern  
11    Vector2D(int x, int y) {  
12        this->x = x;  
13        this->y = y;  
14    }  
15    ...  
16 };
```

3.4.4 Die Initialisierungsliste

Neben der Zuweisung der Attribute im Funktionsrumpf des Konstruktors können wir die Attribute auch als s.g. *Initialisierungsliste* direkt hinter der Deklaration des Konstruktors initialisieren.

Dazu schreiben wir nach der Funktionsdeklaration einen Doppelpunkt `:` und dann die Initialisierung der Attribute in der Form `attribut(wert), ...:`

```
1 class Vector2D {  
2     ...  
3     public:  
4         // default-Konstruktor mit Initialisierungsliste  
5         Vector2D() : x(0), y(0) {}  
6         // Konstruktor mit Initialisierungsliste  
7         Vector2D(int x, int y) : x(x), y(y) {}  
8         ...  
9     };
```

Dies mag auf den ersten Blick etwas verwirrend wirken, aber sobald wir über die Vererbung (siehe Kapitel 4.3) sprechen, wird es mehr Sinn ergeben.

3.4.5 Der default-Konstruktor

Der default-Konstruktor ist ein spezieller Konstruktor, der keine Parameter hat. Er wird automatisch aufgerufen, wenn ein Objekt ohne Parameter erzeugt wird und hat zur Aufgabe, das Objekt auf sinnvolle Werte zu initialisieren.

Falls wir keinen Konstruktor explizit definieren, wird automatisch ein default-Konstruktor vom Compiler generiert. Dieser ruft dann wiederum alle default-Konstruktoren der Member auf (je nach Datentyp des Members unterscheidet sich das Verhalten des default-Konstruktors jedoch, gegen Sie also nicht davon aus, dass er immer alles auf 0 initialisiert).

Sobald wir jedoch einen Konstruktor mit Parametern definieren, wird der implizite default-Konstruktor nicht mehr generiert. In diesem Fall müssen wir den default-Konstruktor explizit definieren.

Vergleichen Sie die folgenden Beispiele:

```
1 class Vector2D {  
2     private:  
3         int x;  
4         int y;
```

```

5 };
6
7 int main() {
8     Vector2D v; // Kein Fehler, impliziter default-Konstruktor wird
   ↳ aufgerufen
9     ...
10 }

```

```

1 class Vector2D {
2 private:
3     int x;
4     int y;
5 public:
6     Vector2D(int x, int y) : x(x), y(y) {}
7 };
8
9 int main() {
10     Vector2D v; // Fehler, impliziter default-Konstruktor existiert nicht
   ↳ mehr, da wir einen Konstruktor definiert haben
11     ...
12 }

```

Es ist empfehlenswert, immer einen default-Konstruktor zu definieren, auch wenn er nur die Attribute auf 0 setzt.

3.4.6 Der Kopierkonstruktor

Ein weitere spezieller Konstruktor ist der *Kopierkonstruktor*. Dieser wird automatisch aufgerufen, wenn ein Objekt durch Kopieren eines anderen Objekts erzeugt wird, also z.B. bei der Übergabe eines Objekts an eine (*call-by-value*-)Funktion oder bei der Rückgabe eines Objekts aus einer Funktion.

Der Kopierkonstruktor hat als Parameter eine Referenz auf ein Objekt der gleichen Klasse:

```

1 class Vector2D {
2     ...
3 public:
4     // Kopierkonstruktor
5     Vector2D(const Vector2D& other) {
6         x = other.x;
7         y = other.y;

```

```

8     }
9     ...
10 };
11 }

```

Im Kopierkonstruktor können wir implementieren, wie ein Objekt kopiert werden soll. In obigem Beispiel kopieren wir einfach die Werte der Attribute des anderen Objekts. Es gibt aber komplexere Anwendungsfälle, z.B. wenn dynamischer Speicher verwendet wird (siehe Kapitel Chapter 6).

Definieren wir keinen expliziten Kopierkonstruktor, wird ein impliziter Kopierkonstruktor vom Compiler generiert. Dieser kopiert einfach die Attribute des anderen Objekts.

3.5 Destruktoren

So wie es den *Konstruktor* zur Initialisierung eines Objekts gibt, gibt es auch den *Destruktor* zum “aufräumen” eines Objekts. Der Destruktor wird automatisch aufgerufen, wenn ein Objekt den Gültigkeitsbereich verlässt bzw. gelöscht wird.

Der Destruktor hat den gleichen Namen wie die Klasse, wird jedoch mit einer Tilde ~ davor geschrieben und hat keinen Rückgabewert und keine Parameter:

```

1 class Vector2D {
2     ...
3 public:
4     ...
5     // (Expliziter) Destruktor
6     ~Vector2D() {
7         std::cout << "Vector2D-Objekt wird gelöscht" << std::endl;
8     }
9 };

```

Wird kein Destruktor explizit definiert, wird ein impliziter Destruktor vom Compiler generiert. Dieser ruft die Destruktoren aller Member-Objekte auf. In vielen Fällen kommt man daher ohne expliziten Destruktor aus. Eine Ausnahme dafür ist z.B., wenn dynamisch allokierte Ressourcen (z.B. mit `new`) innerhalb der Klasse verwendet werden. In diesem Fall wird der Destruktor benötigt, um die Ressourcen wieder freizugeben. Wir werden darauf in Kapitel Chapter 6 genauer eingehen.

3.6 const-Methoden

Genau wie in C, können wir in C++ Variablen als `const` deklarieren. Dies bedeutet, dass wir den Inhalt der Variable nicht verändern dürfen.

Bei primitiven Datentypen (z.B. `int`, `double`, ...) ist dies einfach zu verstehen. Was aber passiert bei Objekten? Haben wir ein Objekt als `const`, bedeutet dies, dass wir die Attribute des Objekts nicht verändern dürfen. Damit der Compiler weiß, dass wir in einer Methode keine Attribute verändern, müssen wir die Methode selbst als `const` deklarieren:

```
1 class Vector2D {
2     ...
3 public:
4     ...
5     void print() const { //< Methode verändert keine Attribute -> const
6         std::cout << "(" << x << ", " << y << ")" << std::endl;
7     }
8     ...
9 };
```

Haben wir einen `const`-Vektor, können wir nun die Methode `print()` aufrufen, da sie als `const` deklariert ist. Es ist auch möglich, zwei Versionen der gleichen Methode zu definieren, eine `const` und eine nicht-`const`. Der Compiler wählt dann automatisch die passende Methode aus, je nachdem, ob das Objekt `const` ist oder nicht.

Methoden, die keine Attribute verändern (d.h. nur *lesend*, nicht aber *schreibend* auf die Attribute zugreifen), sollten immer als `const` deklariert werden!

3.7 Operatorüberladung

In C++ können wir Operatoren für unsere eigenen Klassen überladen. Das bedeutet, dass wir definieren können, was passieren soll, wenn ein Operator (z.B. *ist gleich* `==`, *kleiner* `<` oder *addiere* `+`) auf ein Objekt unserer Klasse angewendet wird.

Für eine Übersicht der möglichen Operatoren und deren Bedeutung siehe die [C++ Referenz](#).

Die Überladung von Operatoren erfolgt durch die Definition einer speziellen Methode, deren Name durch das Schlüsselwort `operator` und den Operator selbst angegeben wird. Zum Beispiel `operator+` für den `+`-Operator oder `operator==` für den `==`-Operator.

Ein Beispiel für die Überladung des `+`-Operators und `==`-Operators für unsere Klasse `Vector2D`:

```

1  class Vector2D {
2  ...
3  public:
4      ...
5      // Addition
6      Vector2D operator+(const Vector2D& other) const {
7          Vector2D result;
8          result.x = x + other.x;
9          result.y = y + other.y;
10         return result;
11     }
12
13     // Gleichheit
14     bool operator==(const Vector2D& other) const {
15         return x == other.x && y == other.y;
16     }
17     ...
18 };
19
20 int main() {
21     Vector2D v1(3, 4);
22     Vector2D v2(1, 2);
23     Vector2D v3 = v1 + v2;
24     v3.print();
25
26     if (v1 == v2) {
27         std::cout << "v1 und v2 sind gleich" << std::endl;
28     }
29
30     ...
31 }

```

4 Klassenhierarchien und Vererbung

Die eigentliche Stärke von C++ bzw. Objektorientierung zeigt sich bei komplexeren Anwendungen. In diesem Kapitel werden wir uns daher mit Klassehierarchien und Vererbung beschäftigen.

4.1 Eigenschaften von Klassen

Ein Programm soll üblicherweise eine bestimmte Aufgabe erfüllen bzw. ein bestimmtes Problem lösen. Bei objektorientierter Programmierung können wir das Problem in Form von Klassen und Objekten modellieren. Bevor wir also für ein gegebenes Problem ein Programm entwickeln, sollten wir uns überlegen, wie wir dies in Klassen abbilden können. Dafür gibt es verschiedene Ansätze.

Stellen Sie sich vor, Sie sollen für ein 2D-Spiel eine Klasse implementieren, die ein Spielobjekt repräsentiert, welches sich auf dem Bildschirm bewegen kann. Nennen wir diese Klasse `GameObject`. Wir können die Anforderungen bzw. Eigenschaften von `GameObject` zuerst einmal umgangssprachlich definieren:

Ein `GameObject`:

- hat eine *Position* auf dem Bildschirm.
- kann sich auf dem Bildschirm bewegen.
- kann auf dem Bildschirm gezeichnet werden.

Wir sehen, dass wir die Position des Objekts verwalten müssen. Da wir uns in einem 2D-Spiel befinden, können wir die Position als Koordinatenpaar (x , y) darstellen, also als zweidimensionalen Vektor. Daraus ergibt sich eine weitere sinnvolle Klasse: `Vector2D`.

Ein `Vector2D`:

- hat eine x - und eine y -Koordinate.
- kann mit einem anderen Vector addiert werden.
- kann mit einem Skalar multipliziert werden.

4.2 UML-Klassendiagramme

Diese Anforderungen können wir nun in Klassenattributen und -methoden überführen. Dabei können wir auf ein s.g. *UML-Klassendiagramm* zurückgreifen. UML steht für *Unified Modeling Language* und ist eine standardisierte Notation zur Modellierung von Software. Ein Klassendiagramm besteht aus Klassen mit Attributen und Methoden. Abbildung 4.1 zeigt ein Klassendiagramm für unsere Klassen `GameObject` und `Vector2D`.

Wir sehen sowohl die Attribute der Klassen (jeweils im oberen Bereich) als auch die Methoden (im unteren Bereich), inklusive ihrer Signatur. Zusätzlich sind die Sichtbarkeiten der Attribute und Methoden angegeben (+ für *public*, - für *private*). Außerdem können wir Beziehungen zwischen den Klassen darstellen. In diesem Fall hat ein `GameObject` eine *Aggregation* zu einem `Vector2D`, dargestellt durch die Linie mit Diamant am Anfang. Dies bedeutet, dass ein `GameObject` ein `Vector2D` hat bzw. *besitzt*.

Ohne an dieser Stelle UML weiter zu erläutern, sehen wir, dass UML-Diagramme nützlich sind, um die Struktur von Klassen zu visualisieren. Sie können auch als Vorlage für die Implementierung dienen. Es gibt viele Tools, um UML-Diagramme zu erstellen. Das obige Diagramm wurde mit [Mermaid](#) erstellt, die Syntax für Klassendiagramme ist [hier](#) dokumentiert.

Auf Basis des Klassendiagramms können wir nun die Klassen implementieren. Dies ist jetzt relativ trivial und sei Ihnen als Übung überlassen.

4.3 Vererbung

Ein weiteres wichtiges Konzept in der objektorientierten Programmierung ist die *Vererbung*. Vererbung bedeutet, dass eine Klasse von einer anderen Klasse *erbt*. Die erbende Klasse übernimmt dabei die Attribute und Methoden der Basisklasse.

Nehmen wir an, wir wollen ein weiteres Spielobjekt implementieren, das den Spieler repräsentiert. Auch dieses Objekt hat eine Position, kann sich bewegen und auf dem Bildschirm gezeichnet werden. Über Vererbung können wir die Klasse `Player` von der Klasse `GameObject` ableiten. Abbildung 4.2 zeigt das erweiterte Klassendiagramm.

Die Beziehung `Player` *erbt von* `GameObject` wird durch die Linie mit Pfeil dargestellt.

Im Code können wir die Vererbung bei der Deklaration der Kindklasse durch den Doppelpunkt und den Namen der Elternklasse (oder Basisklasse) angeben: `class <Kindklasse> : public <Elternklasse> {}`.

Das `public` steht für die Sichtbarkeit der geerbten Attribute und Methoden. Bei behalten wir die Sichtbarkeit der geerbten Elemente bei (d.h. alles, was in der Elternklasse `public` ist, ist es auch in der Kindklasse). `private` würde bedeuten, dass alle geerbten Elemente in der Kindklasse `private` sind und nicht von außen zugreifbar.

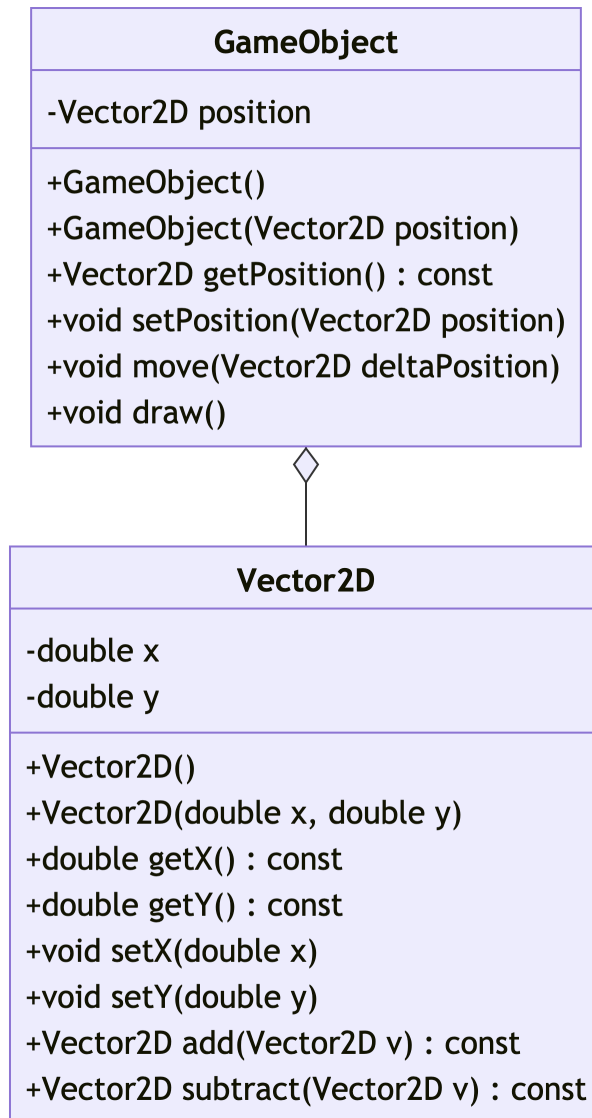


Abbildung 4.1: Klassendiagramm für GameObject und Vector2D

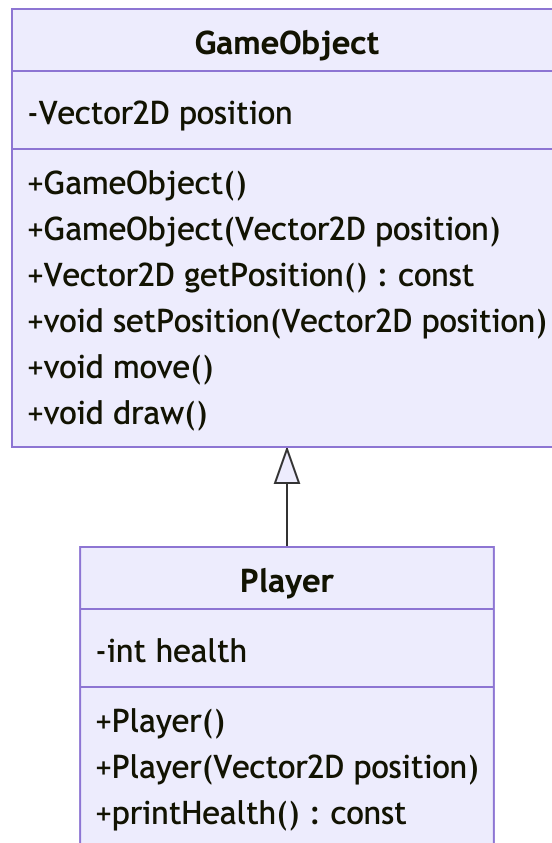


Abbildung 4.2: Player erbt von GameObject

Die vollständige Implementierung der Klasse könnte wie folgt aussehen:

```
1 class Player : public GameObject {
2     private:
3         int health;
4     public:
5         Player() : GameObject(), health(100) {}
6         Player(Vector2D position) : GameObject(position), health(100) {}
7         void printHealth() const {
8             std::cout << "Health: " << health << std::endl;
9         }
10    };
```

Beachten Sie, dass wir in den Konstruktoren der Kindklasse den Konstruktor der Elternklasse explizit aufrufen müssen. Dies geschieht mit dem `:-`-Operator und dem Namen der Elternklasse nach der Deklaration des Konstruktors. Nun ergibt auch die Syntax der Initialisierungsliste (siehe Kapitel 3.4.4) mehr Sinn: Wir rufen dabei einfach die Konstruktoren der jeweiligen Attribute auf.

4.3.1 Auswirkungen von Vererbung

Die Kindklasse erbt die Attribute und Methoden der Elternklasse. Das bedeutet, dass die Kindklasse über alle Attribute und Methoden der Elternklasse verfügt.

Beispielsweise können wir in der Klasse `Player` auf die Methode `draw` zugreifen, obwohl sie in der Klasse `GameObject` definiert ist:

```
1 Player player;
2 player.draw(); // ruft eigentlich GameObject::draw() auf
```

Vererbung kann auch in einer Kette von mehreren Klassen angewendet werden, d.h. eine Klasse `A` kann von einer Klasse `B` erben, die wiederum von einer Klasse `C` erbt. Außerdem kann in C++ eine Klasse von mehreren Klassen gleichzeitig erben.

4.3.2 Sichtbarkeiten bei Vererbung

In Kapitel 3.2 haben wir die Sichtbarkeiten `public` und `private` kennengelernt, die den Zugriff auf Attribute und Methoden einer Klasse regeln. Bei Vererbung gibt es zusätzlich die Sichtbarkeit `protected`. Diese besagt, dass ein Attribut oder eine Methode nur in der Klasse selbst und in abgeleiteten Klassen sichtbar ist.

Um beispielsweise in `Player` direkt auf die Position des Spielers zuzugreifen, müssten wir die Sichtbarkeit von `position` in der Elternklasse auf `protected` setzen:

```
1 class GameObject {
2     protected:
3         Vector2D position;
4     public:
5         ...
6 };
```

Die Sichtbarkeiten in C++ sind:

- `public`: Zugriff von außen.
- `protected`: Zugriff von in der Klasse selbst und in abgeleiteten Klassen.
- `private`: Zugriff nur in der Klasse selbst.

Nutzen Sie immer die restriktivste Sichtbarkeit. Auch bei Vererbung reicht in den vielen Fällen `private` aus.

4.3.3 Virtuelle Methoden und Polymorphie

Eine weitere wichtige Eigenschaft von Vererbung sind *virtuelle Methoden*. Virtuelle Methoden sind Methoden, die in der Elternklasse definiert sind und in der Kindklasse überschrieben werden können. Dies ermöglicht es, dass die Kindklasse eine eigene Implementierung der Methode hat. Dieser Mechanismus wird auch als **Polymorphie** bezeichnet.

Um eine Methode virtuell zu machen, fügen wir das Schlüsselwort `virtual` vor den Methodennamen in der Elternklasse hinzu. In der Kindklasse können wir die Methode dann *überschreiben*, d.h. wir definieren eine Methode mit der gleichen Signatur, die nun bei Objekten des Typs der Kindklasse anstatt der gleichnamigen Funktion der Elternklasse aufgerufen wird.

Nehmen wir an, wir wollen die Methode `move` in der Klasse `GameObject` virtuell machen, damit in der Klasse `Player` eine eigene Implementierung möglich ist, die das Spielerobjekt in Abhängigkeit von der Tastatureingabe bewegt.

```
1 class GameObject {
2     ...
3     public:
4         virtual void move() {
5             // Standardimplementierung
6         }
7     ...
8 };
```

In der Klasse `Player` können wir die Methode dann überschreiben:

```
1 class Player : public GameObject {
2     ...
3     public:
4         void move() override {
5             // Spielerbewegung
6         }
7     ...
8 };
```

Polymorphie bedeutet, dass ein Objekt einer Kindklasse als Objekt der Elternklasse verwendet werden kann, wobei aber die spezifische Implementierung der Kindklasse verwendet wird. Dies ermöglicht es, verschiedene Implementierungen der gleichen Schnittstelle zu verwenden.

Wird die Methode `move` aufgerufen, wird je nach konkretem Objekttyp die entsprechende Implementierung ausgeführt (auch wenn der Zeiger auf das Objekt vom Typ der Elternklasse ist). Das folgende Beispiel verdeutlicht dies:

```
1 Player player;
2 GameObject *obj = (GameObject*) &player; // cast zu GameObject*
3 obj->move(); // ruft Player::move() auf
```

4.3.3.1 Verdecken von Methoden

Hierbei ist zu beachten, dass eine bestimmte Methode in einer Vererbungshierarchie nur dann überschrieben wird, wenn sie in mindestens einer der Kindklassen als `virtual` deklariert ist. Ist dies nicht der Fall, “verdeckt” eine Methode in der Kindklasse die Methode der Elternklasse, statt sie zu überschreiben. Um sicherzustellen, dass eine Methode in einer Kindklasse eine virtuelle Methode der Elternklasse überschreibt, können wir sie optional mit dem Schlüsselwort `override` markieren. In diesem Fall wird ein Compilerfehler erzeugt, wenn die Methode in der Elternklasse nicht virtuell ist. Siehe dazu folgendes Beispiel:

```
1 class GameObject {
2     ...
3     public:
4         void move() {
5             // Achtung: Methode nicht virtuell!!
6         }
7     ...
}
```

```

8 };
9
10 class Player : public GameObject {
11     ...
12     public:
13         virtual void move() {
14             // Methode verdeckt GameObject::move()
15         }
16     ...
17 };

```

Im Beispiel unten wird nun `GameObject::move()` aufgerufen, da die Methode in `GameObject` nicht virtuell ist:

```

1 Player player;
2 GameObject *obj = (GameObject*) &player; // cast zu GameObject*
3 obj->move(); // ruft GameObject::move() auf, da diese Methode nicht virtuell
    ↪ ist

```

Markieren wir die Methode in `Player` als **override**, teilen dem Compiler also mit, dass wir hier eigentlich etwas überschreiben möchten, erhalten wir einen Compilerfehler:

```

1 class Player : public GameObject {
2     ...
3     public:
4         void move() override {
5             // Compilerfehler: Methode überschreibt keine virtuelle Methode
6         }
7     ...
8 };

```

4.3.4 Typcasting

An dieser Stelle ein kleiner Exkurs zum Thema *Typecasting*. Im Beispiel oben haben wir einen Zeiger vom Typ `Player` auf einen Zeiger vom Typ `GameObject` gecastet. Wir haben dazu die klassische C-Syntax verwendet, wobei wir den Zieltypen in Klammern vor den Ausdruck gesetzt haben: `(GameObject*) &player`. Diese Syntax ist weiterhin gültig, allerdings gibt es bessere Möglichkeiten in C++. Das Equivalent zu diesem Ausdruck ist der **static_cast**: `GameObject *obj = static_cast<GameObject*>(&player);`, wobei der Zieltyp in spitzen Klammern angegeben wird.

Der `static_cast` führt keine Überprüfung des Typs durch, d.h. es wird einfach angenommen, dass der Zeiger korrekt gecastet werden kann. Er wird häufig auch für primitive Datentypen verwendet, um z.B. von einem `int` zu einem `double` zu casten.

Nutzen Sie den `static_cast` nur, wenn Sie sicher sind, dass der Zeiger korrekt gecastet werden kann oder für primitive Datentypen.

Im Kontext von Vererbung und Polymorphie ist es jedoch oft sinnvoll, den `dynamic_cast` zu verwenden. Der `dynamic_cast` führt eine Überprüfung des Typs durch und gibt im Fehlerfall `nullptr` zurück. Dies ist besonders nützlich, wenn wir nicht sicher sind, ob der Zeiger tatsächlich auf ein Objekt der Zielklasse zeigt. Der `dynamic_cast` wird wie folgt verwendet: `GameObject *obj = dynamic_cast<GameObject*>(&player);`.

```
1 class GameObject {...};
2 class Player : public GameObject {...};
3 class Enemy : public GameObject {...};
4
5 void isPlayer(GameObject *obj) {
6     Player *player = dynamic_cast<Player*>(obj);
7     if(player) {
8         std::cout << "Objekt ist vom Typ Player" << std::endl;
9     }
10    else {
11        std::cout << "Objekt ist nicht vom Typ Player" << std::endl;
12    }
13 }
14
15 int main() {
16     Player player;
17     Enemy enemy;
18     isPlayer(&player); // Ausgabe: "Objekt ist vom Typ Player"
19     isPlayer(&enemy); // Ausgabe: "Objekt ist nicht vom Typ Player"
20 }
```

Das zur Laufzeit dynamisch ermittelte Objekt wird auch als *dynamischer Polymorphismus* bezeichnet.

Der `dynamic_cast` ist sicherer als der `static_cast`, da er eine Überprüfung des Typs durchführt.

Es gibt noch weitere Casting-Operatoren in C++: Den `const_cast` zum Entfernen von `const`-Qualifizierungen und den `reinterpret_cast` zum Casten von Zeigern auf unterschiedliche Typen. Beide kommen in der Praxis nur in besonderen Anwendungsfällen zum Einsatz seien hier nur zur Vollständigkeit erwähnt.

4.3.5 Abstrakte Klassen

Eine Klasse, die mindestens eine *rein virtuelle* Methode enthält, wird als *abstrakte Klasse* bezeichnet. Eine *rein virtuelle* Methode ist eine Methode, die in der Elternklasse deklariert, aber nicht definiert ist. Sie hat die Form `virtual void method() = 0;`.

Abstrakte Klassen können nicht instanziiert werden, d.h. es können keine Objekte von ihnen erzeugt werden. Sie dienen als Vorlage für konkrete Klassen, die von ihnen erben. Sie können aber als Zeiger oder Referenzen verwendet werden. In der Praxis werden solche Klassen oft als *Interfaces* verwendet, d.h. sie beschreiben eine Schnittstelle, die von konkreten Klassen implementiert werden muss.

4.4 Modultechik

In den bisherigen Beispielen haben wir Klassen gleichzeitig *deklariert* und *definiert* (siehe Kapitel 3.1). Sinnvoll ist es jedoch, die Deklaration und Definition von Klassen zu trennen. Die Deklaration erfolgt in einem Header-File (`.h` oder `.hpp`), die Definition in einer Quelltextdatei (`.cpp`).

Die Deklaration enthält sämtliche Attribute sowie die Deklaration der Methoden, nicht jedoch deren Implementierung. Bei unserer Klasse `Player` könnte die Deklaration in einer Header-Datei `Player.h` so aussehen:

Listing 4.1 `Player.h`

```
1  #pragma once
2  #include "GameObject.h"
3
4  class Player : public GameObject {
5      private:
6          int health;
7      public:
8          Player();
9          Player(Vector2D position);
10         void printHealth() const;
11     };
```

Das `#pragma once` verhindert, dass die Header-Datei mehrfach eingebunden wird.

Die Methoden werden dann separat in einer `.cpp`-Datei implementiert. Dabei besteht der Methodennamen aus dem Klassennamen gefolgt von `::` und dem Methodennamen.

In unserem Beispiel könnte die Datei `Player.cpp` wie folgt aussehen:

Listing 4.2 Player.cpp

```
1  #include "Player.h"
2
3  Player::Player() : GameObject(), health(100) {}
4
5  Player::Player(Vector2D position) : GameObject(position), health(100) {}
6
7  void Player::printHealth() const {
8      std::cout << "Health: " << health << std::endl;
9  }
```

5 Die Standardbibliothek

C++ bietet eine umfangreiche Standardbibliothek mit vielen nützlichen Funktionen und Klassen. Für viele Anwendungsfälle, die in der Praxis auftreten, gibt es bereits eine fertige Lösung in der Standardbibliothek, z.B. für das Verwalten von Daten, das Arbeiten mit Zeichenketten, das Sortieren und finden von Daten etc.

Eine vollständige Übersicht über die Standardbibliothek findet sich in der [C++ Referenz](#).

Header-Dateien der Standardbibliothek werden in der Regel ohne die Dateierweiterung `.h` eingebunden, z.B.:

```
1 #include <iostream> // Ein- und Ausgabe
2 #include <vector>    // Dynamische Arrays
3 #include <string>    // Zeichenketten
4 ...
```

5.1 Container

Für das Verwalten von Daten bietet die Standardbibliothek verschiedene Container-Klassen an. Container sind Klassen, die eine Sammlung von Objekten verwalten. Die Container unterscheiden sich in der Art, wie die Daten gespeichert werden und wie auf die Daten zugegriffen werden kann. Sie verfügen aber über eine einheitliche Schnittstelle, sodass Container unabhängig von ihrer konkreten Implementierung verwendet werden können.

Wichtige Container-Klassen sind: `std::vector`, `std::list` und `std::map`. Es gibt noch weitere Container-Klassen, die wir hier aber nicht behandeln. Eine vollständige Übersicht über die Container-Klassen findet sich in der [C++ Referenz](#).

In Container können beliebige Datentypen gespeichert werden. Die Datentypen müssen aber kopierbar sein, d.h. sie müssen einen Kopierkonstruktor und einen Zuweisungsoperator bereitstellen. Um zu definieren, welcher Datentyp in einem Container gespeichert werden soll, wird der Datentyp in spitzen Klammern angegeben, z.B.: `std::vector<int>`, `std::list<std::string>` usw. Diese Syntax werden uns in Kapitel 7 noch genauer anschauen.

Container haben im Vergleich zu klassischen C-Arrays viele Vorteile: automatische Speicherverwaltung, dynamische Größe, einheitliche Schnittstellen etc. Verwenden Sie in C++ daher immer Container anstelle Arrays.

5.1.1 `std::vector`

`std::vector` ist ein dynamisches Array, das Elemente in einem zusammenhängenden Speicherbereich speichert. Der Zugriff auf die Elemente erfolgt über den Index. Ein `std::vector` kann Elemente am Ende hinzufügen oder entfernen, aber auch auf beliebige Elemente zugreifen oder Elemente an beliebiger Stelle einfügen (letzteres relativ ineffizient, da Elemente umkopiert werden müssen).

```
1  #include <vector>
2
3  int main()
4  {
5      // Ein leeres Array erstellen
6      std::vector<int> v;
7
8      // Elemente hinzufügen
9      v.push_back(1);
10     v.push_back(2);
11     v.push_back(3);
12
13     // Letztes Element entfernen
14     v.pop_back();
15
16     // Elemente ausgeben
17     for (int i = 0; i < v.size(); ++i)
18     {
19         std::cout << v[i] << std::endl;
20     }
21
22     return 0;
23 }
```

Ein `std::vector` verwaltet automatisch im Hintergrund den Speicher für die Elemente. Wenn ein `std::vector` zu klein wird, wird mehr Speicher reserviert und die Elemente in den neuen Speicherbereich kopiert. Seien Sie sich aber bewusst, dass das Kopieren der Elemente Zeit kostet. Wenn Sie wissen, wie viele Elemente Sie speichern möchten, können Sie die Größe des Vektors bereits bei der Erstellung festlegen, um unnötige Kopiervorgänge zu vermeiden: `std::vector<int> v(10);` erstellt einen Vektor mit 10 Elementen.

Ein Vektor ist ein guter Container, wenn Sie eine dynamische Liste von Elementen benötigen, auf die Sie mit einem Index zugreifen möchten.

5.1.2 `std::list`

`std::list` ist eine doppelt verkettete Liste, die Elemente in beliebiger Reihenfolge speichert. Der Zugriff auf die Elemente erfolgt über Iteratoren. Ein direkter Zugriff auf ein bestimmtes Element über den Index ist nicht möglich.

Ein `std::list` kann Elemente am Anfang, am Ende oder an beliebiger Stelle einfügen oder entfernen.

```
1  #include <list>
2
3  int main()
4  {
5      // Eine leere Liste erstellen
6      std::list<int> l;
7
8      // Elemente hinzufügen
9      l.push_back(1);
10     l.push_back(2);
11     l.push_back(3);
12
13     // Erstes Element entfernen
14     l.pop_front();
15     // Letztes Element entfernen
16     l.pop_back();
17
18     // Elemente ausgeben
19     for (auto it = l.begin(); it != l.end(); ++it)
20     {
21         std::cout << *it << std::endl;
22     }
23
24     return 0;
25 }
```

Sie sehen, dass das Interface von `std::list` recht ähnlich zu dem von `std::vector` ist. Einzig der Zugriff auf die Elemente unterscheidet sich.

Eine Liste ist ein guter Container, wenn Sie häufig Elemente einfügen oder entfernen müssen.

5.1.3 Iteratoren

Iteratoren sind Objekte, die den Zugriff auf die Elemente eines Containers ermöglichen. Ein Iterator zeigt auf ein bestimmtes Element im Container und kann verwendet werden, um auf das Element zuzugreifen oder das Element zu verändern. Iteratoren können inkrementiert und dekrementiert werden, um auf das nächste bzw. vorherige Element zu zeigen.

Im Beispiel der Liste oben wird ein Iterator `it` verwendet, um über die Elemente der Liste zu iterieren. Der Iterator wird mit `l.begin()` auf das erste Element gesetzt. Dann wird er in einer Schleife solange inkrementiert, bis er auf das Element `l.end()` zeigt, das hinter dem letzten Element steht.

Zugriff auf das aktuelle Element erfolgt über den Dereferenzierungsoperator `*it`. Der Typ des Iterators hängt vom Container ab, auf den er zeigt. In obigem Beispiel ist der Typ von `it` `std::list<int>::iterator`. Um das Schreiben von langen Typnamen zu vermeiden, kann der Compiler den Typ des Iterators auch automatisch ableiten, indem `auto` verwendet wird.

Eine noch kürzere Schreibweise für die Schleife ist die Verwendung einer **range-based for-Schleife**:

```
1 for (const int& i : l)
2 {
3     std::cout << i << std::endl;
4 }
```

Hierbei erzeugt der Compiler automatisch einen Iterator, der über alle Elemente der Liste iteriert. Der jeweilige Wert, auf den Iterator zeigt, wird in der Schleifenvariable `i` gespeichert. Beachten Sie, dass die Schleifenvariable `i` hier als Referenz deklariert ist, um Kopien der Elemente zu vermeiden (im Falle von trivialen Datentypen wie `int` macht das keinen Unterschied, aber bei komplexeren Datentypen kann das Kopieren Zeit kosten).

Iteratoren sind ein mächtiges Konzept in C++, das es ermöglicht, Container unabhängig von ihrer konkreten Implementierung zu verwenden. Egal welchen Container Sie nutzen, Sie können immer mit Iteratoren auf die Elemente zugreifen.

5.1.4 `std::map`

`std::map` ist eine assoziative Datenstruktur, die Schlüssel-Wert-Paare speichert. Wir können definieren, welche Datentypen für den Schlüssel und den Wert verwendet werden sollen, z.B. `std::map<std::string, int>` für ein string-Schlüssel und integer-Werte. Der Zugriff auf die Elemente erfolgt über den Schlüssel (anstelle über den Index), wie bei einem Array, mit `map["schluessel"]`. Schlüssel müssen in der Map eindeutig sein.

Beim Iterieren über die Elemente bekommt man Paare von Schlüssel und Wert als Objekt vom Typ `std::pair`. Der Schlüssel ist dabei `pair.first` und der Wert `pair.second`.

```
1  #include <map>
2
3  int main()
4  {
5      // Eine leere Map erstellen
6      std::map<std::string, int> m;
7
8      // Elemente hinzufügen
9      m["one"] = 1;
10     m["two"] = 2;
11     m["three"] = 3;
12
13     // Erstes Element ausgeben
14     std::cout << m["one"] << std::endl;
15
16     // Elemente ausgeben
17     for (const auto& p : m)
18     {
19         std::cout << p.first << ": " << p.second << std::endl;
20     }
21
22     return 0;
23 }
```

Eine Map ist ein guter Container, wenn Sie Schlüssel-Wert-Paare speichern möchten und auf die Elemente über den Schlüssel zugreifen möchten.

5.2 Algorithmen

Die Standardbibliothek beinhaltet eine Menge nützlicher Algorithmen an. Die Meisten davon arbeiten mit Sequenzen von Elementen, weshalb sie mit Containern nutzbar sind. Einige Beispiele für Algorithmen sind: `std::sort` und `std::find`.

Die Sequenz, auf der ein Algorithmus arbeitet, wird durch zwei Iteratoren definiert, die den Anfang und das Ende der Sequenz angeben. Daher können die Algorithmen mit allen Containern verwendet werden, die Iteratoren unterstützen.

Die Algorithmen sind in der Header-Datei `<algorithm>` definiert. Eine vollständige Übersicht findet sich in der [C++ Referenz](#).

5.2.1 std::sort

`std::sort` sortiert die Elemente einer Sequenz. Der Algorithmus arbeitet *in-place*, d.h. er sortiert direkt die angegebene Sequenz und verwendet den QuickSort-Algorithmus, der im Durchschnitt eine Laufzeit von $O(n \log n)$ hat.

```
1  #include <algorithm>
2  #include <vector>
3
4  int main()
5  {
6      std::vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5};
7
8      // Vektor sortieren
9      std::sort(v.begin(), v.end());
10
11     // Elemente ausgeben
12     for (const auto& i : v)
13     {
14         std::cout << i << std::endl;
15     }
16
17     return 0;
18 }
```

`std::sort` funktioniert mit allen Datentypen, für die eine *kleiner-als*-Relation definiert ist, d.h. bei denen definiert ist, wie zwei Elemente des Datentyps verglichen werden können. Für eingebaute Datentypen wie `int`, `float`, `double` etc. ist das bereits definiert. Für eigene Datentypen müssen Sie ggf. einen Vergleichsoperator definieren (siehe Kapitel 3.7).

Alternativ können Sie auch eine eigene Vergleichsfunktion angeben, wenn Sie eine spezielle Sortierreihenfolge benötigen:

```
1  #include <algorithm>
2
3  // Vergleichsfunktion: Nimmt zwei Elemente und gibt zurück,
4  // ob das erste Element größer ist als das zweite
5  bool compare(int a, int b)
6  {
7      return a > b;
8  }
9
```

```

10 int main()
11 {
12     std::vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5};
13
14     // Vektor sortieren mit eigener Vergleichsfunktion
15     std::sort(v.begin(), v.end(), compare);
16
17     ...
18
19     return 0;
20 }

```

5.2.2 std::find

`std::find` sucht ein Element in einer Sequenz. Der Algorithmus gibt einen Iterator zurück, der auf das erste Vorkommen des gesuchten Elements zeigt. Wenn das Element nicht gefunden wird, wird der Iterator `end()` zurückgegeben.

```

1  #include <algorithm>
2
3  int main()
4  {
5      std::vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5};
6
7      // Element suchen
8      auto it = std::find(v.begin(), v.end(), 4);
9
10     if (it != v.end())
11     {
12         std::cout << "Element gefunden: " << *it << std::endl;
13     }
14     else
15     {
16         std::cout << "Element nicht gefunden" << std::endl;
17     }
18
19     return 0;
20 }

```

6 Dynamischer Speicher

Wie in C auch, können wir C++ zur Laufzeit Speicher dynamisch allokiert und wieder freigegeben werden. Dies werden wir in diesem Abschnitt genauer betrachten.

6.1 Statischer vs. dynamischer Speicher

In allen bisherigen Beispielen haben wir statischen Speicher verwendet. Das bedeutet, dass der Speicher zur Kompilierzeit reserviert wird und zur Laufzeit nicht mehr verändert werden kann.

Legen wir eine Variable oder ein Objekt an, so weiß der Compiler, wieviel Speicher benötigt wird und kann das Reservieren des benötigten Speichers **statisch** in den erzeugten Maschinencode einfügen. Legen wir Variablen innerhalb einer Funktion an, so wird der Speicher für diese Variablen zwar zur Laufzeit beim Aufrufen der Funktion auf dem s.g. *Stack* reserviert und, nach verlassen der Funktion, wieder freigegeben, es handelt sich aber dennoch um statischen Speicher.

```
1 void foo()
2 {
3     // Speicher für a und v wird statisch auf dem Stack reserviert
4     int a = 5;
5     Vector2D v;
6 }
7
8 int main()
9 {
10     foo();
11 }
```

Dynamische Speicherallokation hingegen, erlaubt es uns zur Laufzeit Speicher zu reservieren und wieder freizugeben, ohne dass der Compiler im Vorfeld wissen muss, wieviel Speicher benötigt wird. Dies ist besonders nützlich, wenn wir zur Laufzeit entscheiden wollen, wieviel Speicher benötigt wird, oder wenn wir Variablen in einer Funktion anlegen wollen, die über den Funktionsaufruf hinaus bestehen bleiben soll.

In C hatten wir dafür die Funktionen `malloc` und `free` zur Verfügung. In C++ funktioniert das ganze etwas anders.

6.2 Speicher allozieren und freigeben

Für dynamische Speicheroperationen stehen die Operatoren `new` und `delete` zur Verfügung. Der Operator `new` allokiert Speicher für ein Objekt und gibt einen Zeiger auf den allokierten Speicher zurück. Der Operator `delete` gibt den allokierten Speicher wieder frei.

```
1 int* p = new int;    // allokiert Speicher für ein int
2 *p = 5;              // schreibt 5 in den allokierten Speicher
3 delete p;            // gibt den allokierten Speicher wieder frei
```

`new` und `delete` rufen jeweils den Konstruktor bzw. Destruktor des allokierten Objekts auf. Existiert ein Konstruktor mit Parametern, können wir diesen bei `new` auch entsprechend aufrufen.

```
1 #include <iostream>
2
3 class Vector2D
4 {
5 private:
6     int x, y;
7
8 public:
9     Vector2D(int x, int y) : x(x), y(y) { std::cout << "Konstruktor
   ↳ aufgerufen" << std::endl; }
10    ~Vector2D() { std::cout << "Destruktor aufgerufen" << std::endl; }
11 }
12
13 int main()
14 {
15     Vector2D* pV = new Vector2D(1, 2); // Ausgabe: "Konstruktor aufgerufen"
16     delete pV;                          // Ausgabe: "Destruktor aufgerufen"
17 }
```

6.3 Smart Pointer

Ein Problem bei der Verwendung von `new` und `delete` ist, dass wir den Speicher explizit freigeben müssen. Vergessen wir das, so entsteht ein s.g. **Memory Leak**, d.h. der allokierte

Speicher bleibt auch nach der eigentlichen Verwendung der Variablen reserviert und steht nicht mehr zur Verfügung. Passiert das häufiger (z.B. in einer Schleife), so wird der Speicher des Computers immer knapper und das Programm wird schlussendlich abstürzen bzw. der Computer nicht mehr benutzbar sein.

```
1  int main()
2  {
3      int *p;
4
5      while (true)
6      {
7          p = new int;
8          // Speicher wird nicht freigegeben -> Memory Leak
9      }
10
11     delete p;
12 }
```

Memory Leaks sind **immer** ein Fehler im Programm und sollten unbedingt vermieden werden.

In komplexeren Anwendungen treten Memory Leaks allerdings häufig *versteckt* auf und es ist nicht trivial, sie zu finden und zu beheben. Um das Problem zu umgehen, gibt es in C++ die s.g. **Smart Pointer**. Smart Pointer sind Klassen, die den Speicher automatisch freigeben, sobald der Pointer nicht mehr benötigt wird. Es gibt verschiedene Arten von Smart Pointern, die sich in ihrem Verhalten unterscheiden.

Allen Varianten gemein ist, dass sie mit s.g. *reference counting* arbeiten. Das bedeutet, dass der Smart Pointer zählt, wie oft eine Referenz auf das Objekt existiert. Wird der Smart Pointer kopiert, so wird der Zähler erhöht, wird der Smart Pointer gelöscht, so wird der Zähler verringert. Erreicht der Zähler den Wert 0, so wird der allokierte Speicher freigegeben.

Smart Pointer sind im Header `<memory>` definiert und befinden sich im Namespace `std`.

Bevor wir uns die verschiedenen Smart Pointer ansehen, benötigen wir noch eine kurze Einführung in das Konzept der *Ownership*.

6.3.1 Ownership bzw. *wer besitzt das Objekt?*

Existieren mehrere Zeiger bzw. Referenzen auf das gleiche Objekt, so stellt sich die Frage, wer für das Löschen des Objekts verantwortlich ist. Ist dies nicht klar geregelt, so kann es zu schwerwiegenden Fehlern kommen, wenn ein Objekt mehrfach gelöscht wird oder an einer Stelle im Programm gelöscht wird, obwohl das Objekt an anderer Stelle noch verwendet wird.

Aus diesem Grund gibt es in C++ das Konzept der *Ownership*. *Ownership* bedeutet, dass ein Objekt einem bestimmten Besitzer zugeordnet ist, der für das Löschen des Objekts verantwortlich ist. Wird der Besitzer gelöscht, so wird auch das Objekt gelöscht. Smart Pointer implementieren dieses Konzept, indem sie den Besitzer eines Objekts darstellen. Mittels der s.g. *Move Semantics* kann der Besitzer eines Objekts auch übertragen werden (siehe weiter unten).

6.3.2 `std::shared_ptr`

Ein `std::shared_ptr` ist ein Smart Pointer, der s.g. *shared ownership* implementiert. Das bedeutet, dass ein Objekt mehreren `std::shared_ptr` zugeordnet werden kann und der Speicher erst freigegeben wird, wenn der letzte `std::shared_ptr` gelöscht wird.

Einen `std::shared_ptr` deklarieren wir mittels `std::shared_ptr<T>`, wobei T der Typ des Objekts ist, auf das der `std::shared_ptr` zeigt. Im Konstruktor übergeben wir den Zeiger auf das Objekt, das der `std::shared_ptr` verwalten soll, z.B. `std::shared_ptr<int> p(new int(5));`

Auf den Inhalt des `std::shared_ptr` greifen wir (wir bei regulären Zeigern) mittels des Dereferenzoperators `*` zu, z.B. `*p = 5;`.

```
1  #include <memory>
2  #include <iostream>
3
4  int main()
5  {
6      int *pInt = new int(5);
7
8      {
9          std::shared_ptr<int> p1(pInt); // p1 zeigt auf das Objekt pInt
10
11         {
12             std::shared_ptr<int> p2 = p1; // p2 zeigt auf das selbe Objekt
13             // p2 wird am Ende des Blocks gelöscht, aber das Objekt bleibt
14             // erhalten (da p1 noch existiert)
15         }
16
17         std::cout << *p1 << std::endl; // Ausgabe: 5
18         // Nun wird p1 gelöscht. Da p1 der letzte shared_ptr ist, wird auch
19         // das Objekt gelöscht.
```

```

20 // pInt zeigt auf ein bereits gelöschttes Objekt -> undefined behavior
21 std::cout << *pInt << std::endl;
22
23 return 0;
24 }

```

`std::shared_ptr` ist die am Einfachsten zu verwendende Art von Smartpointern. Allerdings ist *shared ownership* häufig nicht die Semantik, die Sie eigentlich benötigen. Falsche Verwendung kann dazu führen, dass Objekte nicht freigegeben werden (falls z.B. irgendwie in die “hintersten Ecken” des Programms noch eine Referenz auf ein Objekt existiert, obwohl das Objekt eigentlich schon gar nicht mehr verwendet wird). In den meisten Fällen ist es daher besser, *unique ownership* zu verwenden.

Vermeiden Sie es, `std::shared_ptr` zu verwenden, wenn Sie nicht wirklich *shared ownership* benötigen.

6.3.3 `std::unique_ptr`

Ein `std::unique_ptr` ist ein Smart Pointer, der s.g. *unique ownership* implementiert. Das bedeutet, dass ein Objekt nur einem `std::unique_ptr` zugeordnet werden kann. Wird der `std::unique_ptr` gelöscht, so wird auch das Objekt gelöscht. Der Besitz des Objekts ist daher eindeutig geregelt und es kann zu keinen “Mißverständnissen” kommen, da der Compiler die korrekte Verwendung von `std::unique_ptr` sicherstellen kann.

Ein `unique_ptr` wird wie ein `shared_ptr` deklariert, nur dass wir `std::unique_ptr<T>` statt `std::shared_ptr<T>` verwenden.

Soll das Objekt einem anderen `std::unique_ptr` zugeordnet werden, so können wir das Objekt mittels *Move Semantics* übertragen. Das dazu verwendete Schlüsselwort ist `std::move`, z.B. `std::unique_ptr<int> p2 = std::move(p1);`.

Unten sehen Sie ein Beispiel, welches die verschiedenen Eigenschaften von `std::unique_ptr` illustriert.

```

1 #include <memory>
2 #include <iostream>
3
4 int main()
5 {
6     std::unique_ptr<int> p1(new int(5));
7
8     std::unique_ptr<int> p2;

```

```

9
10     p2 = p1; // Compilerfehler: unique_ptr kann nicht kopiert werden (dies
    ↪ würde zu "shared ownership" führen)
11
12     p2 = std::move(p1); // p2 zeigt nun auf das Objekt, das zuvor p1
    ↪ zugeordnet war
13                               // p1 verliert das Objekt und zeigt nun auf nullptr
14     std::cout << *p2 << std::endl; // Ausgabe: 5
15
16     p2.reset(new int(10)); // p2 zeigt nun auf ein neues Objekt, das alte
    ↪ Objekt wird gelöscht
17     std::cout << *p2 << std::endl; // Ausgabe: 10
18
19     return 0;
20 }

```

Ein häufiges Pattern bei der Verwendung von `std::unique_ptr` ist, dass wir auf ein von einem `unique_ptr` verwaltetes Objekt zugreifen wollen, ohne das Objekt zu übernehmen (z.B. in einer Funktion). Dazu können wir eine `const`-Referenz auf das Objekt übergeben, z.B.:

```

1  #include <memory>
2  #include <iostream>
3
4  void foo(const std::unique_ptr<int>& p)
5  {
6      std::cout << *p << std::endl;
7  }
8
9  int main()
10 {
11     std::unique_ptr<int> p(new int(5));
12     foo(p);
13
14     return 0;
15 }

```

In den meisten Anwendungsfällen ist *unique ownership* und damit `std::unique_ptr` die beste Methode.

6.3.4 `std::make_unique` und `std::make_shared`

Um das Erstellen von `std::unique_ptr` und `std::shared_ptr` zu vereinfachen, gibt es die Funktionen `std::make_unique` und `std::make_shared`. Dabei werden die Parameter an den Konstruktor des Objekts weitergegeben.

Anstatt also `std::unique_ptr<Vector2D> p(new Vector2D(1.,2.));` zu schreiben, können wir auch `std::unique_ptr<Vector2D> p = std::make_unique<Vector2D>(1.,2.);` verwenden. Eine weitere Vereinfachung stellt das Schlüsselwort `auto` dar, mit dem wir den Typ des Pointers nicht explizit angeben müssen. Der Ausdruck vereinfacht sich dann zu `auto p = std::make_unique<Vector2D>(1.,2.);`.

Nutzen Sie `auto` nur dann, wenn der Typ des Pointers klar ist und der Code dadurch lesbarer wird. Vermeiden Sie `auto` bei trivialen Typen wie `int`, `float`, etc.

6.3.5 `std::dynamic_pointer_cast` und `std::static_pointer_cast`

Um von einem `std::shared_ptr` auf einen anderen Typ zu casten, können wir die Funktionen `std::dynamic_pointer_cast` und `std::static_pointer_cast` verwenden. Diese Funktionen verhalten sich wie die entsprechenden C++-Casts (siehe Kapitel 4.3.4), nur dass sie mit `std::shared_ptr` arbeiten.

```
1  #include <memory>
2
3  class A {
4  public:
5      virtual void foo(){};
6  };
7  class B : public A {};
8
9  int main()
10 {
11     std::shared_ptr<A> pA = std::make_shared<A>();
12     std::shared_ptr<B> pB = std::dynamic_pointer_cast<B>(pA);
13
14     return 0;
15 }
```

7 Templates

Templates sind weiteres mächtiges Feature von C++, welches die Wiederverwendung von Code erleichtert und dabei hilft, doppelten Code zu vermeiden. Sie haben bereits Templates verwendet (siehe z.B. Kapitel [5.1](#) und [6.3](#)).

In diesem Kapitel werden wir uns damit beschäftigen, wie Templates funktionieren und wie sie in eigenen Klassen und Funktionen verwendet werden können.

7.1 Was sind Templates?

Templates sind eine Art von generischen Funktionen oder Klassen, die mit verschiedenen Datentypen oder Konstanten arbeiten können. Templates sind in der Lage, Code zu generieren, der auf verschiedenen Datentypen basiert. Dies ermöglicht es, Funktionen und Klassen zu schreiben, die mit beliebigen Datentypen arbeiten können, ohne dass der Code für jeden Datentyp neu geschrieben werden muss. Das Template dient also als Bauplan bzw. Schablone für die Erstellung von Funktionen oder Klassen.

Betrachten Sie folgendes Beispiel:

```
1 class Point2D {
2 private:
3     int x;
4     int y;
5 public:
6     Point2D(int x, int y) : x(x), y(y) {}
7     int getX() const { return x; }
8     int getY() const { return y; }
9 };
```

Hier haben wir wieder eine einfache Klasse `Point2D`, die eine 2D-Koordinate repräsentiert. Der Typ der Koordinaten ist `int`. Was aber, wenn wir eine Klasse `Point2D` haben möchten, die mit `double`-Koordinaten arbeitet? Dazu müssten wir eine neue Klasse `Point2DDouble` schreiben, die genau so aussieht wie `Point2D`, aber `double`-Koordinaten verwendet. Es leuchtet ein, dass dies nicht sehr effizient ist. Hier kommen Templates ins Spiel.

7.2 Template-Klassen

Mit Templates können wir eine generische Klasse schreiben, die mit verschiedenen Datentypen arbeiten kann. Unsere `Point2D`-Klasse könnte also so aussehen:

```
1  template <typename T>
2  class Point2D {
3  private:
4      T x;
5      T y;
6  public:
7      Point2D(T x, T y) : x(x), y(y) {}
8      T getX() const { return x; }
9      T getY() const { return y; }
10 };
```

Wir haben das Schlüsselwort `template` vor der Klassendeklaration verwendet, um eine Template-Klasse zu erstellen. Der Ausdruck `<typename T>` definiert einen Typparameter `T`, der als Platzhalter für den tatsächlichen Datentyp verwendet wird. In diesem Fall kann `T` durch jeden beliebigen Datentyp ersetzt werden. Die Klasse `Point2D` kann also mit `int`, `double`, `float` oder anderen Datentypen verwendet werden. Es können auch mehrere Typparameter definiert werden, z.B. `<typename T1, typename T2>`.

Um eine Instanz der Klasse `Point2D` zu erstellen, müssen wir den Datentyp angeben, den wir verwenden möchten. Zum Beispiel:

```
1  Point2D<int> point1(1, 2);
2  Point2D<double> point2(1.5, 2.5);
```

Der Compiler wird dann den entsprechenden Code für die Klasseninstanz `Point2D<int>` und `Point2D<double>` generieren, indem er den Typparameter `T` durch den angegebenen Datentyp ersetzt und den entstandenen Code kompiliert. Es ist wichtig zu beachten, dass der Compiler für jeden konkrete Typisierung eine eigene Klasse generiert. Das bedeutet, dass die Klassen `Point2D<int>` und `Point2D<double>` zwei verschiedene Klassen sind, die nichts miteinander zu tun haben (auch wenn sie auf den selben Template basieren). Es können z.B. keine `Point2D<int>`-Objekte in eine `Point2D<double>`-Variable kopiert werden.

Da Template-Klassen erst bei der Verwendung mit dem tatsächlichen Datentyp kompiliert werden, werden sie normalerweise komplett in Header-Dateien definiert, um sicherzustellen, dass der Compiler die komplette Definition sehen kann, wenn sie benötigt wird.

Eine Aufteilung in Header- und cpp-Datei ist auch möglich, hat aber ein paar Tücken und wird hier nicht weiter behandelt.

7.3 Template-Funktionen

Neben Template-Klassen können auch Template-Funktionen erstellt werden. Template-Funktionen sind Funktionen, die mit verschiedenen Datentypen oder Konstanten arbeiten können. Die Syntax für die Definition einer Template-Funktion ist ähnlich wie bei Template-Klassen, nur dass der Typparameter in diesem Fall als Platzhalter für den Datentyp der Funktion dient.

Hier ist ein einfaches Beispiel:

```
1  template <typename T>
2  T add(T a, T b) {
3      return a + b;
4  }
5
6  int main() {
7      int sum1 = add(1, 2);
8      double sum2 = add(1.5, 2.5);
9      return 0;
10 }
```

8 Verschiedenes

In diesem Abschnitt werden einige weitere Themen angesprochen, die für die Arbeit mit C++ relevant sind.

8.1 Exceptions

Exceptions sind ein Mechanismus, um Fehlerbehandlung zu realisieren. Sie ermöglichen es, Fehlerbedingungen zu signalisieren und in einem anderen Teil des Programms zu behandeln.

In C wird oft ein Fehlerzustand durch den Rückgabewert einer Funktion signalisiert. Dieser Rückgabewert wird dann von der aufrufenden Funktion überprüft und entsprechend behandelt. Dieses Vorgehen hat jedoch einige Nachteile: Wir mißbrauchen den Rückgabewert der Funktion (wir verwenden ihn nicht, um das Ergebnis der Funktion zu übergeben, sondern um den Fehlerzustand zu signalisieren) und die Fehlerbehandlung muss direkt in der aufrufenden Funktion stattfinden. Auch ist schwierig, komplexere Fehler zu behandeln, da wir nur einen einzelnen Rückgabewert haben.

8.1.1 Exceptions in C++

Exceptions werden in C++ mit dem Schlüsselwort **throw** geworfen und mit **try** und **catch** behandelt. Ein **throw**-Statement erzeugt ein Objekt, das den Fehlerzustand beschreibt. Dieses Objekt wird dann durch die Aufrufkette der Funktionen hindurch gereicht, bis es von einem **catch**-Block abgefangen wird.

```
1  #include <iostream>
2  #include <stdexcept>
3
4  void foo() {
5      throw std::runtime_error("Fehler in foo");
6  }
7
8  void bar() {
9      foo();
10 }
```

```

11
12 int main() {
13     try {
14         bar();
15     } catch (const std::runtime_error& e) {
16         std::cerr << "Fehler: " << e.what() << std::endl;
17     }
18 }

```

In diesem Beispiel wird in der Funktion `foo` eine `std::runtime_error`-Exception geworfen. Diese wird dann nicht in der aufrufenden Funktion `bar` behandelt, sondern in der `main`-Funktion. Dort wird eine Fehlermeldung ausgegeben. Der Typ des Exceptions-Objekts bestimmt, welcher `catch`-Block ausgeführt wird. In diesem Beispiel wird nur ein `catch`-Block verwendet, der alle `std::runtime_error`-Exceptions abfängt. Es können jedoch auch mehrere `catch`-Blöcke hintereinander verwendet werden, um verschiedene Exceptions zu behandeln.

`std::exception` ist die Basisklasse für alle Exceptions in C++. Sie definiert eine Methode `what()`, die eine Beschreibung des Fehlers zurückgibt.

Im Header `stdexcept` sind weitere Standard-Exception-Klassen definiert, die von `std::exception` erben:

```

namespace std {
    class logic_error;
    class domain_error;
    class invalid_argument;
    class length_error;
    class out_of_range;
    class runtime_error;
    class range_error;
    class overflow_error;
    class underflow_error;
}

```

Es macht Sinn, diejenige Exception-Klasse zu verwenden, die am besten zum Fehler passt. Wir können auch eigene Exception-Klassen definieren, die von `std::exception` erben.

References

Mahr, Thomas. 2023. *Achtung c++: C++ Und Objektorientierung Wirklich Verstehen*. 3. Ausgabe. Thomas Mahr. <https://opac.th-nuernberg.de/TouchPoint/start.do?Query=+1035%3D%22BV048675705%22+IN+%5B2%5D&View=fhn&Language=de&Branch=>.